Project Part 1

AI Personal Finance Assistant

Functional Specification

Group G

Sofia Ursachi

Aidan Gillooley

Adam Hayden

Ulysse Couchoud

Jamie Ocallaghan

0. Group Members:

Student name	Student ID	Main responsibility worked on	Approximate contribution %
Aidan Gillooley	A00030921	Preliminary Schedule	100%
Adam Hayden	A23448102	Proof-reading and Functional Requirements	100%
Ulysse Couchoud	A00133493	System Architecture	100%
Jamie Ocallaghan	A23365913	Functional Requirements	100%
Sofia Ursachi	A00035832	Introduction and General Description	100%

1. Table of Contents

0. Group Members: *1*

2. Introduction *3*

2.1 Overview: 4

2.2 Business Context: 4

3. General Description *5*

3.1 User Characteristics and Objectives: 6

3.2 Quality Requirements: 6

3.3 Constraints & Assumptions: 7

4. Functional Requirements *7*

4.1 Product / System Functions: 8

4.2 Functional Requirements List: 8

5. System Architecture *14*

5.1 Initial Architecture Considerations: 15

5.2 Draft Architecture Overview: 15

6. Preliminary Schedule *17*

6.1 Project Plan: 18

6.2 Gantt Chart: 18

6.3 Task Dependencies and PERT Chart: 19

6.4 JIRA Backlog: 21

6.5 Workload Distribution: 23

6.6 Schedule Risks: 24

2. Introduction

2.1 Overview:

Many people find it difficult to keep track of where their money goes. Spending is spread across debit cards, credit cards, subscriptions and apps such as Revolut, and most banking apps only show a list of transactions without explaining spending habits or helping users plan. Keeping a manual budget in a spreadsheet is time-consuming, and many people give up after a few weeks.

The AI Personal Finance Assistant (AIPFA) is a web application that brings a user's financial information together in one place and helps them understand and manage it. Users create a secure account and record their income and expenditure, either by entering transactions manually or by importing transaction data from supported sources such as bank CSV exports. The system automatically sorts each transaction into a spending category (for example, food, transport, shopping, or bills), and users can correct the category if needed. Users can also view, search, filter, edit, and delete their transaction history.

Building on this data, the system lets users set budgets per category, view statistics and trends through visual dashboards, and receive AI-generated insights, such as summaries of their spending, warnings when they are close to a budget limit and personalised suggestions on where they could save money.

The system will interact with the following external systems:

- **Bank and file sources:** Transaction data is imported from files exported by banks (e.g. CSV) and, as a future extension, from Open Banking providers.
- **External LLM API:** A third-party large language model is used by a separate AI service to generate insights and recommendations. Only anonymised or aggregated data is sent to it.

2.2 Business Context:

The system is intended to be developed and operated by a small fintech start-up targeting the Irish and wider EU market. Its main customers are students, young professionals, and other individuals who want better control of their personal finances but do not want to pay for a financial advisor or spend time maintaining spreadsheets.

The business goals are:

- **Attract and retain users** by offering a simple, low-effort way to track spending that is more useful than a standard banking app.
- **Generate revenue** through a freemium model: core features (transaction tracking, categorisation and basic budgets) are free, while advanced AI insights, unlimited imports and detailed reports are part of a paid subscription.
- **Build user trust** by handling financial data securely and in compliance with GDPR, which is essential for a finance product.
- **Differentiate from competitors** through AI features that explain spending in plain language rather than just displaying numbers.

The proposed system supports these goals directly. Automatic import and categorisation reduce the effort required from users, which helps retention; the AI insights provide the premium value behind the subscription model; and the security and privacy design decisions described in Section 5 support user trust and regulatory compliance.

3. General Description

3.1 User Characteristics and Objectives:

The system has two main types of users:

Regular users (primary). Members of the public, mostly aged 18–35, such as students and young working adults. They are comfortable using web and mobile apps and online banking but are not expected to have any knowledge of accounting, budgeting methods or financial terminology. Many have irregular income (for example from part-time jobs) and many small transactions, and they want to spend as little time as possible managing their finances.

System administrators (secondary). Members of the development and operations team who manage the system, maintain the default list of categories, and monitor system health. They have strong technical expertise but do not have access to users' individual financial data.

From the user's perspective, the main objectives and requirements are to:

- Register and log in securely, with confidence that their data is private.
- Get their transactions into the system quickly, ideally by importing rather than typing them.
- See where their money goes, broken down by category, with minimal manual work.
- Fix mistakes easily, such as a wrong category, a wrong amount or a duplicate entry.
- Set budgets and be warned before overspending.

3.2 Quality Requirements:

QR01 — Security: All financial and personal data must be protected. Passwords are hashed, all communication uses HTTPS, and users can only ever access their own data.

QR02 — Performance: Common actions such as loading the transaction history, searching or filtering should respond within 2 seconds for a user with up to 10,000 transactions. Importing a typical monthly bank statement should be completed within a few seconds.

QR03 — Scalability: The system should handle a growing number of users and transactions, including peaks in usage at the start of each month and around payday, without a significant drop in performance.

QR04 — Usability: A new user with no financial background should be able to register, add or import transactions and view their spending breakdown without needing a tutorial. The interface should work on both desktop and mobile browsers.

QR05 — Maintainability: The system should be modular, so that individual components, in particular the AI service and the LLM provider, can be changed or replaced without major changes to the rest of the system.

3.3 Constraints & Assumptions:

Constraints

- **GDPR:** As the system processes the personal and financial data of EU users, it must comply with GDPR, including lawful processing, data minimisation, the user's right to access and delete their data, and storing data within the EU where possible.
- **Financial regulation:** Direct access to bank accounts through Open Banking (PSD2) requires a licensed provider, so the initial version is limited to file-based imports. The AI must also not give regulated financial or investment advice, only general spending insights.
- **Third-party LLM:** The system depends on an external LLM provider's availability, pricing, rate limits and terms of use. Sensitive identifying data cannot be sent to it.
- **Bank export formats:** Import support is limited by the file formats banks provide, which differ between banks and may change without notice.
- **Budget:** The project has a very limited budget, so it must rely mainly on free and open-source technologies and keep LLM API costs low.
- **Time and team:** The system is developed by a five-person student team within one academic semester.

Assumptions

- Users have internet access and a modern web browser.
- Users can download transaction files (e.g. CSV) from their online banking.
- Transactions are mainly in euro; multi-currency support is not required in the first version.
- An affordable LLM API will be available for the duration of the project.

4. Functional Requirements

4.1 Product / System Functions:

The AI Personal Finance Assistant is designed to help users manage their personal finances all in one place. Its primary purpose is to make it easier for users to keep a record of their income and expenditure without having to do so manually. Users can create an account and then securely access their financial information. They have the option of adding transactions themselves or importing transaction data from supported sources. The system will save these transactions and associate them with the appropriate user account. It will also automatically sort the transactions into various spending categories, for example, food, transport, shopping and bills. If the system has incorrectly categorised a transaction, users will be able to alter the category themselves. Users can also view their past transactions and search or filter them using details such as date, amount or category. This transaction information will later be used by the budgeting, statistics and AI features of the application.

The main functions dealt with in this section are:

- creating a user account and logging in.
- adding transactions manually.
- importing transaction data, storing transactions.
- automatically categorising transactions.
- changing transaction categories, viewing transaction history.
- searching and filtering transactions.
- editing transactions and deleting transactions.
- password reset
- budgets and warnings
- statistics and dashboards
- ai summaries
- anonymisation before LLM calls
- account deletion

4.2 Functional Requirements List:

The following functional requirements list the main features that are needed for users to manage their transactions, accounts, budgets, AI insights and data protection.

FR01 — User Account Creation and Login

The system must enable users to set up an account and log in securely. A user's financial information will be associated with their individual account.

Criticality: High

Technical Issues: The login details of users must be stored securely, and the system must ensure that other users do not have access to someone else's financial information.

Dependencies: This function is required for the other transaction features since transactions must be linked to a specific user.

FR02 — Adding Transactions

The system must allow users to add financial transactions manually. Users should be able to input details such as the amount, date and description. The transaction can then be automatically categorised by the system.

Criticality: High

Technical Issues: The system should verify that the information entered by the user is valid before saving the transaction.

Dependencies: This feature depends on FR01 because the transaction needs to be linked to the user's account.

FR03 — Importing Transaction Data

The system must allow users to import transaction data from supported external sources. The transactions that are imported will be included in the user's transaction history.

Criticality: High

Technical Issues: The system must be capable of dealing with the format of the imported data and it should, where possible, avoid adding the same transaction more than once.

Dependencies: This feature depends on FR01. Additionally, it may depend on the external source being used and on the format of the data that it provides.

FR04 — Storing Transactions

The system must store the users' financial transactions so that they can be seen and used by the other parts of the application.

Criticality: High

Technical Issues: It is necessary for the transaction information to be stored accurately and securely. The system also has to ensure that users can only access their own transactions.

Dependencies: It depends on FR01 and supports the other requirements related to transactions.

FR05 — Automatic Transaction Categorisation

The system should automatically assign the appropriate categories to transactions based on the information available about them. Examples of categories are food, transport, shopping, entertainment and bills.

Criticality: High

Technical Issues: The system might not always identify the correct category, particularly when the description of the transaction is unclear. The categorisation method must therefore be tested to confirm that it produces useful results.

Dependencies: It relies on transaction information being available via FR02, FR03 and FR04.

FR06 — Changing Transaction Categories

The system should allow users to change the category of a transaction if it has been incorrectly categorised.

Criticality: Medium

Technical Issues: The system must correctly save the new category without altering any of the other details of the transaction.

Dependencies: It depends on FR04 and FR05.

FR07 — Viewing Transaction History

The system must enable users to see their past transactions. The transaction history should include useful details such as the date, description, amount and category.

Criticality: High

Technical Issues: The system has to show the right transactions for the logged-in user and display the information in a clear manner.

Dependencies: It depends on FR01 and FR04.

FR08 — Searching and Filtering Transactions

The system should permit users to search and filter their transactions. Users should be able to search or filter transactions using details such as date, category, amount or description.

Criticality: Medium

Technical Issues: The searches and filters should function correctly even if a user has many transactions.

Dependencies: It depends on FR04 and FR07.

FR09 — Editing Transactions

The system should allow users to edit the details of an existing transaction, for example the amount, date, description or category.

Criticality: Medium

Technical Issues: The system needs to verify that the new information is valid before saving the changes.

Dependencies: It depends on FR01, FR04 and FR07.

FR10 — Deleting Transactions

The system should allow users to delete transactions which were entered incorrectly or which are no longer needed.

Criticality: Medium

Technical Issues: The system must ensure that users are able to delete only their own transactions. A confirmation should also be given to reduce the possibility of accidentally deleting a transaction.

Dependencies: It depends on FR01 and FR04.

FR11 – Logout and Password Reset

The user shall be able to log out. This will close the session of the client and require the user to login in again to access the application. A user who forgets their password should be able to reset it through a time-limited link sent via email.

Criticality: Medium

Technical Issues: Reset links should be single use so that the link could not be used maliciously if gotten by a different user.

Dependencies: FR01

FR12 – Category Budgets

The user shall be able to set a monthly budget for each category and see it compared to their actual spending in that category for that specific month.

Criticality: High

Technical Issues: spending is calculated from stored transactions; this means that anytime anything from the database is changed, the budget and spending must be calculated again

Dependencies: FR01, FR04, FR05.

FR13 – Budget Warnings

The system shall warn the user if 80% of the budget is used and again if 100% of the budget is used in each category.

Criticality: High

Technical Issues: spending is shown in the app and as a notification; It triggers for each category once, once a month

Dependencies: FR12.

FR14 – Statistics and Dashboards

The system shall show the users spending by category and by month, income versus outcome, monthly trends using different data visualisation techniques such as pie charts, line graphs and bar charts.

Criticality: High

Technical Issues: The database needs to be organised so that data is easily representable and navigable.

Dependencies: FR04, FR05.

FR15 – AI Spending Summary and Suggestions

The system shall generate messages for the user of different statistics and suggestions (e.g. "You spend 20% more on takeaway this month") that the user can make informative decisions off. All results should inform the user that it's general information and not financial advice.

Criticality: High

Technical Issues: If the LLM is having issues this feature won't be working/available, the system should inform the user of the error and if user agreed to terms and conditions of sending data to us. We will receive an error code and be able to review it.

Dependencies: FR14, FR16.

FR16 – Data anonymisation for the LLM

Before any call to the LLM, the system shall remove any personally identifying data and send only aggregated figures.

Criticality: High

Technical Issues: names, emails, passwords, bank details, locations and account numbers should never leave the system. Tests should be put in place to ensure this.

Dependencies: FR14. Supports QR01 and QR05

FR17 – Account Deletion

The user shall have the choice to delete their account and all data stored relating to that account. They should also have the option to request the data stored about them in a CSV file.

Criticality: High

Technical Issues: Required for GDPR. User should be prompted for a password to delete to ensure accidental clicks don't happen since it cannot be undone.

Dependencies: FR01, FR04, Supports QR01

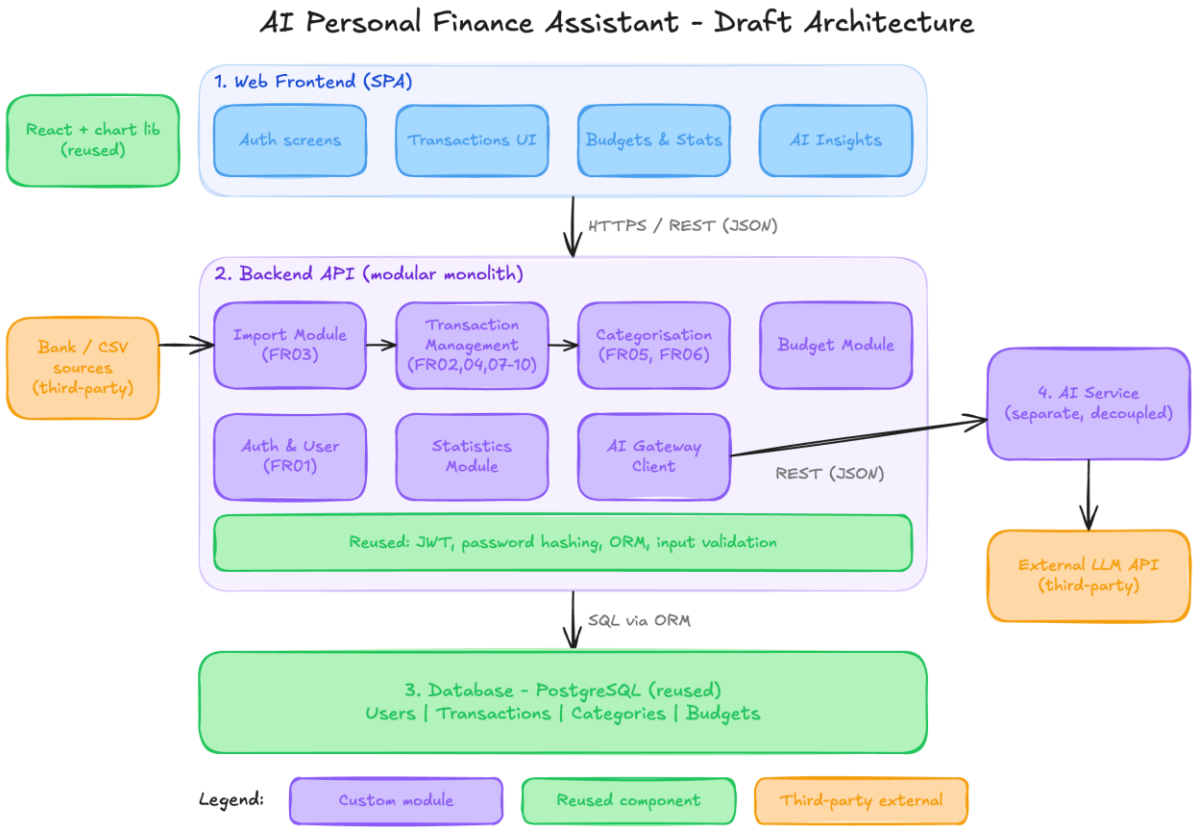
5. System Architecture

5.1 Initial Architecture Considerations:

- **Financial data protection:** The system will handle sensitive financial information, therefore authentication, access control and secure data storage will be important architectural concerns.
- **Scalability and performance:** The architecture should be able to support an increasing number of users and financial transactions while maintaining acceptable performance.
- **AI integration:** The AI component should be integrated with the application without tightly coupling it to the core financial services. This should allow the AI component to be modified or replaced more easily in the future.

5.2 Draft Architecture Overview:

The system follows a layered architecture made of four main parts: a web frontend, a backend API structured as a modular monolith, a relational database, and a separate AI service.



Component	Responsibility	Related FRs
Web Frontend (SPA)	Registration and login screens, transaction forms, history with search and filters, category correction, budget and statistics dashboards, AI insights	UI for FR01–FR17
Auth & User Module	Account creation, secure login, session/token handling, per-user access control	FR01, FR11, FR17
Transaction Management Module	Create, store, view, search/filter, edit and delete transactions with validation; ownership checks	FR02, FR04, FR07, FR08, FR09, FR10
Import Module	Parse supported formats (e.g. CSV/OFX or bank data), validate rows, detect duplicates, pass transactions to the Transaction module	FR03
Categorisation Module	Automatic category assignment (keyword rules first, with an optional ML/LLM fallback for unclear descriptions); saves manual corrections	FR05, FR06
Budget Module	Budgets per category and period, comparison with actual spending	FR12, FR13
Statistics Module	Aggregations and trends over transactions	FR14
AI Gateway Client	Sends anonymised, relevant financial data to the AI service and returns results to the API	FR15, FR16
AI Service	Analysis and personalised recommendations; the only component that talks to the LLM	FR15
Database	Users, transactions, categories, budgets	FR01 - FR17

Reused and third-party components

- **Reused (libraries, frameworks, open source):**
  - a frontend framework such as React for reusing components;
  - Shadcn components for frontend and many other libraries (like Bklit UI for statistics, Aceternity UI, React Bits etc...);
  - PostgreSQL;
  - an ORM;
  - JWT and password hashing libraries (e.g. bcrypt);
  - CSV/XML parsers.
- **Third-party external:**
  - the **external LLM API** used by the AI service;

- the **bank or file sources** feeding the Import module.

Key design decisions

1. **Security.** Passwords are hashed and never stored in plain text. Every query is filtered by the authenticated user's ID, so users cannot see each other's data.
2. **Scalability.** The backend is stateless (token-based), so it can be scaled horizontally. Indexes on date, category and amount keep searching and filtering fast on large histories.
3. **Privacy toward the LLM.** Only the minimum necessary data is sent (aggregated or anonymised), with no identifying details.

6. Preliminary Schedule

6.1 Project Plan:

The project is planned over a 13-week semester using an iterative, Scrum-style approach. The first three weeks (W1–W3) are used for the functional specification (Part 1), which is the document presented here. The remaining ten weeks are organised into five two-week sprints: Sprint 0 for design and set-up, and Sprints 1–4 for implementation, integration and testing. Weeks are numbered relative to the start of the project; they will be mapped to the module's actual submission dates once these are confirmed.

Work is tracked in JIRA (Section 6.4). The team meets weekly to review progress and plans each sprint at the start of the sprint; each sprint ends with a short review and a demo of the working increment. A story is considered done when it is implemented, covered by tests, reviewed by another team member through a pull request, and merged into the main branch. The architecture, technology choices and communication mechanisms are refined in Sprint 0 (Part 2), so this plan is preliminary and will be updated at that point.

Phase	Weeks	Main activities	Milestone
1. Specification	W1–W3	Requirements, user stories, quality requirements, functional requirements FR01–FR17, draft architecture and this schedule (T1)	M1: Functional specification (Part 1) submitted
2. Design & set-up (Sprint 0)	W4–W5	Architecture, database schema and API contract; wireframes; repository, CI and JIRA set-up (T2, T3)	M2: Architecture and API contract agreed; environment ready
3. Core build (Sprint 1)	W6–W7	Auth and transaction management, frontend screens and the start of the AI service (T4, T5, T8)	M3: Register, log in and manage transactions end-to-end
4. Features (Sprint 2)	W8–W9	Import, categorisation and search; budgets and statistics; frontend and AI service completed (T5–T8)	M4: Import, categorisation, budgets and statistics working
5. Dashboards & AI (Sprint 3)	W10–W11	Dashboards, budget screens and AI insights integrated (T9)	M5: Feature complete
6. Testing & delivery (Sprint 4)	W12–W13	System testing, security and performance review, final report and demo (T10)	M6: Tested system, final report and demo delivered

The order of the sprints follows the dependencies between the functional requirements in Section 4: user accounts (FR01) and transaction storage (FR02, FR04, FR07, FR09, FR10) come first because every other feature depends on them; import, categorisation and search (FR03, FR05, FR06, FR08) follow; and dashboards and AI insights, which consume the transaction data, are built last. The AI service is started early, in parallel with the core build, because it depends on the external LLM provider and carries the most technical uncertainty.

6.2 Gantt Chart:

The Gantt chart below shows the planned start and end of each task. Tasks outlined in red are on the critical path, and grey bars show the slack available to other tasks. The diamonds mark the milestones M1–M6.



Figure 1: Preliminary Gantt chart (weeks W1–W13)

ID	Task	Weeks	Duration	Lead
T1	Requirements & functional specification (Part 1)	W1–W3	3 wk	All
T2	Architecture, database schema & API contract	W4–W5	2 wk	Ulysse
T3	UI wireframes & development environment set-up	W4–W5	2 wk	Sofia
T4	Auth & transaction management (FR01, 02, 04, 07, 09, 10)	W6–W7	2 wk	Aidan, Jamie
T5	Frontend: auth, transaction and history screens	W6–W8	3 wk	Sofia
T6	Import, categorisation & search (FR03, 05, 06, 08)	W8–W9	2 wk	Adam
T7	Budgets & statistics modules	W8–W9	2 wk	Aidan
T8	AI service & LLM gateway client	W6–W8	3 wk	Ulysse
T9	Dashboards, budget screens & AI insights	W10–W11	2 wk	Ulysse, Sofia
T10	System testing, final report & demo	W12–W13	2 wk	All

6.3 Task Dependencies and PERT Chart:

The PERT chart shows the dependencies between tasks. Each task is shown with its duration, its earliest start (ES) and earliest finish (EF), measured in weeks from the start of the project, and its slack, which is how many weeks it can be delayed without delaying the project. All dependencies are finish-to-start.

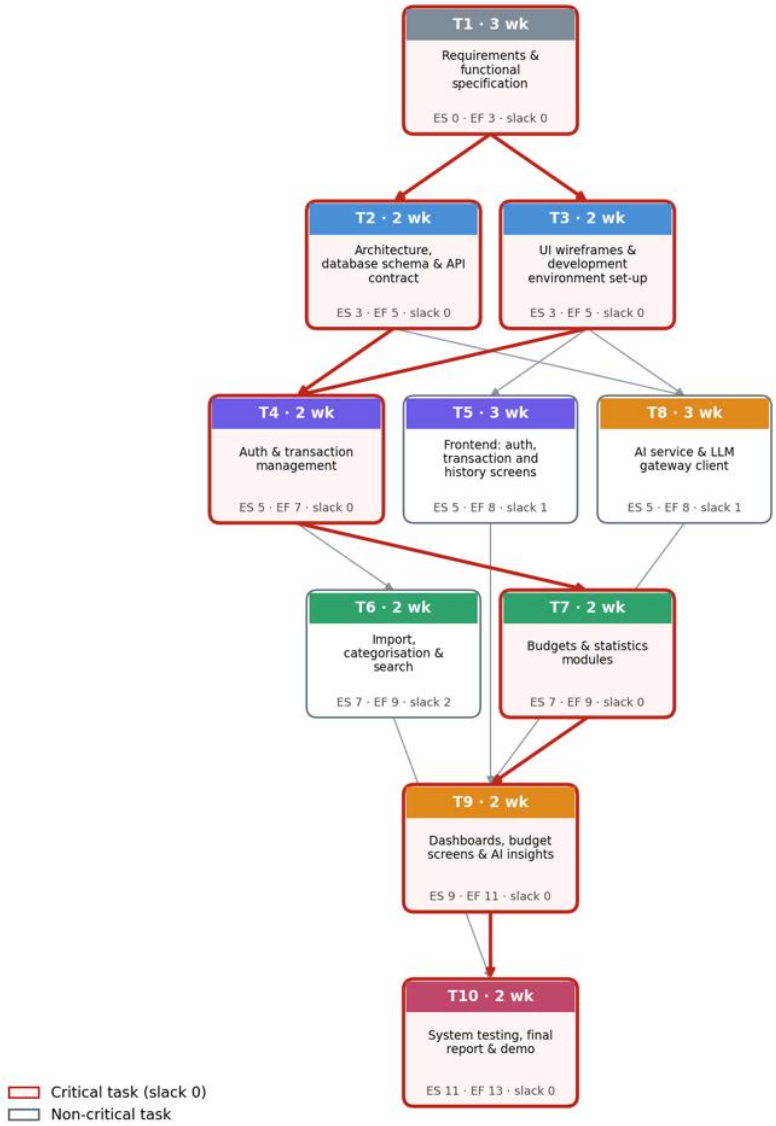


Figure 2: PERT network with critical path in red

Critical path. The tasks with zero slack (T1, T2, T3, T4, T7, T9, T10) form the critical path of 13 weeks: the specification, then the design tasks, then auth and transaction management, then the budgets and statistics modules, then the dashboards and AI insights, and finally testing and the final report. Any delay in these tasks delays the final delivery, so they are scheduled first in each sprint. The remaining tasks have one or two weeks of slack.

ID	Predecessors	Duration	ES	EF	Slack	Critical
T1	–	3 wk	0	3	0	Yes
T2	T1	2 wk	3	5	0	Yes
T3	T1	2 wk	3	5	0	Yes
T4	T2, T3	2 wk	5	7	0	Yes

T5	T3	3 wk	5	8	1	No
T6	T4	2 wk	7	9	2	No
T7	T4	2 wk	7	9	0	Yes
T8	T2, T3	3 wk	5	8	1	No
T9	T5, T7, T8	2 wk	9	11	0	Yes
T10	T6, T9	2 wk	11	13	0	Yes

Two dependencies are worth noting. First, import, categorisation, search, budgets and statistics (T6, T7) all wait for T4, because they require authenticated users and stored transactions (FR01, FR04). Second, the AI service (T8) only needs the architecture and API contract (T2) and the environment (T3), so it runs in parallel with the core build and is joined to the statistics data in T9.

6.4 JIRA Backlog:

The product backlog is managed in a JIRA Scrum project (key PFA). Work is organised into eight epics, broken down into user stories and a small number of technical tasks. Stories are prioritised using MoSCoW (Must, Should, Could) and estimated in story points using a Fibonacci-style scale. The board uses the columns To Do, In Progress, In Review and Done. The backlog below contains 23 items totalling 124 story points.

Epic	Name	Related requirements	Items	Story points
E1	Accounts & security	FR01, QR01	1	5
E2	Transactions	FR02, FR04, FR07–FR10	4	20
E3	Import	FR03	1	8
E4	Categorisation	FR05, FR06	1	5
E5	Budgets	FR12, FR13	1	8
E6	Statistics & dashboard	FR14	2	13
E7	AI insights	FR15, FR16, QR05	5	26
E8	Platform, design & quality	QR01–QR05	8	39

Sprint	Weeks	Items	Goal	Story points
Sprint 0	W4–W5	3	Design and set-up: architecture, schema, API contract, wireframes, environment	13
Sprint 1	W6–W7	5	Users can register, log in and manage transactions	28

Sprint 2	W8–W9	7	Import, categorisation, search/filter, statistics API, AI service core	41
Sprint 3	W10–W11	3	Budgets, dashboards and AI insights integrated (feature complete)	19
Sprint 4	W12–W13	5	Testing, security review, GDPR features, final report and demo	23

Backlog items (SP = story points):

Key	Epic	Item	Related	Priority	SP	Sprint	Assignee
PFA-1	E8	Set up repository, CI pipeline and Docker development environment	–	Must	3	0	Aidan
PFA-2	E8	Create database schema, API contract (OpenAPI) and architecture decisions, including LLM provider choice	FR01–FR10, QR05	Must	5	0	Ulysse
PFA-3	E8	Produce wireframes for all main screens (mobile and desktop)	QR04	Must	5	0	Sofia
PFA-4	E1	As a user, I want to register and log in securely (JWT) and see only my own data	FR01	Must	5	1	Aidan
PFA-5	E2	As a user, I want to add a transaction manually, with validation, and have it stored securely	FR02, FR04	Must	5	1	Jamie
PFA-6	E2	As a user, I want to view, edit and delete my transactions so that I can correct mistakes	FR07, FR09, FR10	Must	5	1	Jamie
PFA-7	E8	Build auth screens, transaction form and list in the React frontend	FR01–FR10	Must	8	1	Sofia
PFA-8	E7	Create AI service skeleton with a provider-independent LLM interface	QR05	Must	5	1	Ulysse
PFA-9	E3	As a user, I want to import a bank CSV file, with duplicate detection, so that I do not have to type transactions	FR03	Must	8	2	Adam
PFA-10	E4	As a user, I want transactions categorised automatically (keyword rules) and to change a category if it is wrong	FR05, FR06	Must	5	2	Adam
PFA-11	E2	As a user, I want to search and filter by date, category, amount and description	FR08	Should	5	2	Jamie
PFA-12	E5	As a user, I want monthly budgets per category, compared with actual spending, with a warning near the limit	FR12, FR13	Must	8	2	Aidan
PFA-13	E6	Provide statistics endpoints (spending by category and by month)	FR14	Must	5	2	Adam
PFA-14	E7	Anonymise and aggregate data before it is sent to the LLM	QR01, FR16	Must	5	2	Ulysse
PFA-15	E2	Build history UI with search, filters and category correction	FR06–FR10	Must	5	2	Sofia
PFA-16	E6	Build budget screens and dashboard charts (spending by category, monthly trend)	FR14	Must	8	3	Sofia
PFA-17	E7	As a user, I want an AI summary of my spending and general saving suggestions in the app	FR15	Must	8	3	Ulysse
PFA-18	E7	Build AI insights panel in the frontend	FR15	Should	3	3	Sofia

PFA-19	E8	Security review (HTTPS, hashing, access control) and account deletion for GDPR	QR01, FR17	Must	5	4	Aidan
PFA-20	E8	Performance test with 10,000 transactions (under 2 s) and mobile usability check	QR02, QR04	Should	3	4	Adam
PFA-21	E8	End-to-end testing and bug fixing	All	Must	5	4	Jamie
PFA-22	E8	Final report, demo preparation and hand-over	–	Must	5	4	All
PFA-23	E7	As a user, I want to ask questions about my spending in a chat box	FR15	Could	5	4	Ulysse

Sprint 2 is the heaviest sprint. If velocity is lower than planned, the only “Could” item (PFA-23) is dropped first, followed by the “Should” items, so that all “Must” items are delivered.

6.5 Workload Distribution:

The work is shared so that each member leads one area of the system and contributes to testing and documentation. The effort is estimated in person-days, where one person-day is about four working hours. The total estimated effort is 145 person-days, which is roughly 2.2 person-days (about 9 hours) per member per week over 13 weeks. The leads below follow the areas of responsibility in Part 1 where these exist and can be adjusted by the team in Part 2.

Member	Focus	Lead tasks	Person-days	Share
Sofia	Frontend & UX lead	T3 wireframes, T5 frontend screens, T9 dashboards (with Ulysse)	30	20%
Jamie	Transactions & API lead	T4 transaction management (with Aidan), T2 API contract support	28	20%
Ulysse	Architecture & AI lead	T2 architecture and schema, T8 AI service, T9 AI insights	31	20%
Aidan	Security, auth & DevOps lead	T3 environment and CI, T4 auth, T7 budgets, security review	29	20%
Adam	Data lead	T6 import, categorisation and search, T7 statistics, performance testing	27	20%

Estimated person-days per task and team member:

ID	Task	Sofia	Jamie	Ulysse	Aidan	Adam	Total
T1	Requirements & functional specification	4	4	4	4	4	20
T2	Architecture, database schema & API contract	–	4	5	2	–	11
T3	UI wireframes & development environment set-up	7	–	–	3	–	10
T4	Auth & transaction management	–	9	–	8	–	17
T5	Frontend: auth, transaction and history screens	10	4	–	–	–	14

T6	Import, categorisation & search	–	3	2	–	13	18
T7	Budgets & statistics modules	–	–	–	8	4	12
T8	AI service & LLM gateway client	–	–	10	–	2	12
T9	Dashboards, budget screens & AI insights	6	–	7	–	1	14
T10	System testing, final report & demo	3	4	3	4	3	17
	Total	30	28	31	29	27	145

Every task has a named lead who is accountable for it, and most tasks have a second member who reviews the work. Specification (T1) and testing, the final report and the demo (T10) are shared by the whole team. Pull-request reviews are rotated so that nobody is the only person who understands a module.

6.6 Schedule Risks:

Risk	Impact	Mitigation
Slippage on the critical path	Delays final delivery, because tasks on the critical path have no slack	Schedule critical tasks first in each sprint; review progress weekly; cut “Could” and “Should” items before moving the deadline
LLM provider cost, rate limits or outage	AI insights unavailable or too expensive	Provider-independent AI service; cache results; send only aggregated data; mock LLM for development and testing
Bank CSV formats differ or change	Imports fail or contain wrong data	Start with Revolut and one Irish bank; use per-bank import profiles; validate rows and report errors to the user
Overloaded Sprint 2	Stories carried over to later sprints	Prioritise with MoSCoW; use the slack on T6; re-plan at each sprint planning meeting
Limited team availability (assignments, exams)	Lower velocity than planned	Keep a shared calendar; avoid single points of failure through pair reviews; maintain a buffer in Sprint 4
Privacy or GDPR issues	Sensitive data exposed or non-compliant processing	Anonymise data before the LLM call; per-user access checks; security review in Sprint 4